

Required Corrections for PROJECT_PLAN-3

AI-Driven Adaptive Multi-Objective CPU–GPU Workload Scheduling and Resource Optimization for Cloud HPC

Purpose: a practical correction guide for the current project plan and implementation. It identifies what must be changed before continuing profiling, ML, scheduling, and optimization. It preserves the project’s core idea rather than redesigning it.

Basis: the current project plan together with the validated implementation history already established for Rodinia 3.1, BFS, and CFD. Where implementation evidence is still pending, this document marks the item as pending rather than treating an assumption as a result.

1. Overall assessment

The architecture is substantially correct. The main problem is not the research direction; it is the precision of the measurement layer and the boundary between **pre-execution prediction features** and **post-execution measurements**. Fix the profiling/data model first. Do not add more advanced ML or optimization algorithms until the measured dataset is internally consistent.

Priority	Area	Action
P0	Timing semantics	Fix CFD parsers, inspect source timing, validate BFS total-path definition.
P0	Dataset separation	Separate raw runs, aggregated performance, and pre-execution ML/scheduler data.
P0	ML leakage	Remove CPU/GPU execution times and measured transfer times from classifier inputs.
P1	Scheduler	Define where predicted/lookup execution estimates come from and normalize objectives consistently.
P1	Energy/cost	Label proxies and modeled cloud costs correctly; document assumptions and source/date.
P1	Hybrid/DAG	Define partitioning rules and clarify that the project constructs the scheduling DAG.
P2	Evaluation	Use workload-aware validation and treat expected results as hypotheses.

2. CFD corrections

2.1 Fix the GPU parser

The plan uses a pattern equivalent to **seconds per iteration = number**. The verified CFD output is the opposite order: **number seconds per iteration**.

Correct direction: `r"([\d.]+)\s+seconds per iteration"`

Do not finalize the parser until the source inspection confirms exactly what the printed value represents.

2.2 Fix the CPU parser

The plan uses a BFS-style **Compute time:** parser for CFD CPU. That output has not been established as the CFD CPU timing line. Do not reuse a parser merely because another Rodinia workload prints a similar phrase.

Create a CFD-specific parser from the actual *euler3d_cpu* output, after one successful CPU run and source/output inspection.

2.3 Inspect timing semantics before changing the dataset

The current CFD GPU measurement reports seconds per iteration. The source must be inspected to determine whether timing covers kernel execution only, synchronization, transfers, or another benchmark-defined interval. Record the semantics explicitly in the dataset.

Validated CFD GPU runs currently exist, but timing semantics remain a gating item. Do not call the value “kernel time” unless source inspection proves that.

2.4 Correct CFD build assumptions

Rodinia 3.1 contains legacy CUDA assumptions. The original CFD binary expected **libcudart.so.4**. The executable therefore failed on the modern Kaggle runtime. Rebuilding for the Tesla T4 with the available CUDA toolchain was required, and a missing **helper_cuda.h** header also had to be resolved.

```
Validated rebuild approach:
nvcc -O2 -Xptxas -v -arch=sm_75 -I/kaggle/working/Rodinia/cuda/hybridsort euler3d.cu -o euler3d
```

The final plan should describe this as a workload-specific compatibility repair, not as a universal Rodinia build command.

3. BFS corrections

3.1 Do not double-count GPU time

BFS GPU phase timing includes repeated synchronous CUDA operations/copies inside the traversal loop. Therefore, a universal formula such as **GPU total = GPU execution + H2D + D2H** can double-count work.

Use a workload-specific, source-validated “GPU total path time” definition. For BFS, the current measurement should retain separate fields for GPU phase, initial H2D phase, final D2H phase, and the validated total path definition.

3.2 Correct the BFS command

```
CPU: ./bfs 4 ../../data/bfs/graph1MW_6.txt
GPU: ./bfs ../../data/bfs/graph1MW_6.txt
```

The GPU command does not include the CPU thread-count argument.

3.3 Remove misleading fixed input-size claims

The plan’s fixed 28 MB BFS example is incorrect for the current **graph1MW_6.txt** input, which is approximately 62 MB. Compute input size programmatically from the actual file used.

3.4 Remove generic BFS timing regexes

BFS CPU output is of the form **Compute time: 0.037191**. The instrumented GPU output is of the form **GPU Kernel Time: ... ms**. These are workload-specific parsers, not a generic “Time:” rule.

3.5 Keep current BFS results but label them correctly

The recent repeated runs establish a useful profiling baseline: CPU benchmark time is roughly 35.5 ms; GPU phase is roughly 3.7 ms; initial H2D is roughly 14.8 ms; final D2H is roughly 0.7 ms; the current validated GPU total path is roughly 19.25 ms. These values should not be presented as a universal formula or as ML samples. The H2D instrumentation also includes host initialization between copies, so calling it “pure transfer time” would be inaccurate.

4. Timing data model — required redesign

Create three layers instead of one mixed table.

Layer	One row means	Contains	Used for
Raw execution dataset	One actual run	CPU/GPU timings, failures, parser status, run number, config	Reliability and measurement analysis
Aggregated performance dataset	One workload + input configuration	Median/mean/std and validated CPU/GPU comparable measurements	Ground truth and scheduler estimates
ML/scheduler dataset	One pre-execution workload/task instance	Static workload/device/context features + target label	Prediction and scheduling

Six repetitions are repetitions of one configuration, not six independent ML samples. Do not inflate the ML sample count by treating repeated runs as independent workloads.

4.1 Timing fields should be semantically explicit

- `benchmark_reported_time_ms` — the workload’s own reported timing, if any.
- `cpu_process_wall_time_ms` — complete process wall time, retained for diagnostics but not automatically treated as CPU execution time.
- `gpu_phase_time_ms` — workload-specific GPU phase; definition must be documented.
- `gpu_kernel_time_ms` — only if kernel-only timing is actually instrumented and validated.

- `host_to_device_time_ms` / `device_to_host_time_ms` — measured transfer intervals, with instrumentation scope documented.
- `gpu_total_path_time_ms` — only after a workload-specific definition prevents double counting.

5. ML dataset and leakage corrections

This is one of the most important corrections. A classifier that chooses CPU/GPU before execution cannot use quantities that only become known after executing both devices.

Remove from pre-execution classifier inputs: `cpu_execution_time`, `gpu_execution_time`, measured H2D/D2H time, total transfer time derived from measurement, measured CPU/GPU ratio, and any other post-run timing quantity.

Use as ground truth/evaluation: measured CPU/GPU performance, validated speedup, preferred device, energy/cost measurements, and runtime outcomes.

5.1 Separate features by availability

Feature class	Examples	Allowed before execution?
Static workload	workload, input size, algorithm class, estimated operations	Yes
Static hardware	CPU core count/frequency, GPU model/memory	Yes
Derived static estimates	estimated parallelism, memory intensity, compute intensity	Yes, if rubric is reproducible
Runtime context	scheduler-managed ready-task count, current resource pressure	Yes, if available to scheduler
Post-execution measurements	CPU/GPU time, measured transfer time, measured energy	No for the pre-execution classifier

5.2 Make expert-derived features reproducible

Features such as `estimated_parallelism`, `estimated_memory_intensity`, and `estimated_compute_intensity` are acceptable only if the project specifies a reproducible scoring/rubric. Otherwise they become subjective labels.

Similarly, `estimated_transfer_size_mb` must be derived from static input/data-movement information, not from the measured transfer duration.

6. Label construction and validation

The target **preferred_device** is a measured/descriptive ground-truth label, not an ML prediction. It should be created only when CPU and GPU timing definitions are genuinely comparable for that workload.

If the timing semantics are incomparable, keep the label as UNKNOWN and fix the measurement rather than forcing a class.

6.1 Workload-aware validation

A random train/test split can leak workload identity when many records belong to the same workload/configuration. Use workload-aware validation where feasible, such as leave-one-workload-out evaluation. Because the number of distinct workloads is still small, report instability rather than hiding it.

Report per-workload results and macro-averaged metrics. Do not imply broad generalization from a small benchmark set.

7. ML model architecture correction

A classifier predicts the device class but does not automatically provide the execution-time values required by a multi-objective scheduler. The architecture therefore needs an explicit timing-estimate source.

Practical staged design: start with the classifier plus validated aggregated lookup values for candidate execution-time estimates. If data volume becomes sufficient, add regression models that predict CPU time, GPU time, or speedup.

Do not add XGBoost, deep learning, reinforcement learning, or other advanced models merely for novelty before the measurement dataset is stable.

8. Scheduler corrections

The scheduler should follow this order:

```
Pre-execution features
↓
ML prediction (classifier; optional regression)
↓
Candidate CPU / GPU / Hybrid
↓
Dependency + transfer + runtime-resource analysis
↓
Multi-objective scoring
↓
Final assignment
↓
Actual execution
↓
Measured outcome + feedback
```

8.1 Remove circular execution-time logic

If the scheduler objective contains execution time, define exactly where each candidate estimate comes from: validated lookup, regression model, or a documented benchmark-derived estimate. Do not compute a “prediction” using the actual execution time of the candidate that has already run.

8.2 Resource pressure terminology

On Kaggle, there is no production cloud GPU queue available to the project. Use terms such as **scheduler-managed ready/queued task count** or **resource-pressure proxy**, and define the proxy mathematically.

9. Transfer-aware scheduling

Keep transfer awareness as a project contribution, but distinguish static data size from measured transfer duration.

- Use static estimated transfer size before execution when making a pre-execution decision.
- Use measured transfer duration for profiling, model fitting, and feedback.
- For a task graph, account for predecessor output availability and the data movement required between CPU and GPU assignments.
- Do not assume that a single transfer formula applies to every Rodinia workload.

10. Multi-objective optimization corrections

The weighted-sum idea is valid, but every objective needs a precise definition and consistent normalization.

Typical objectives in the plan are execution time, cost, energy, utilization/resource penalty, and waiting time. Define each quantity and normalize all objectives to a common scale before applying weights.

10.1 Resource penalty

Do not leave `resource_penalty` as an undefined generic term. Initially it can be a documented proxy based on current scheduler-managed resource pressure. Later it can incorporate measured waiting delay or utilization.

10.2 Dynamic weights

Dynamic weighting needs explicit measurable trigger conditions. For example, a rule may increase the weight of energy when an energy constraint is active, or increase waiting-time importance when queued-task delay crosses a documented threshold. Avoid vague “adaptive weights” with no trigger definition.

10.3 Deadlines

The current plan mentions deadline proximity in dynamic weighting, but deadlines are not otherwise part of the project. Remove that term unless deadlines are explicitly added to the task model.

10.4 NSGA-II

Keep NSGA-II as an optional/advanced extension. The weighted-sum implementation is sufficient for the first complete end-to-end system. Do not introduce NSGA-II until the objectives and data are stable.

11. Energy model corrections

If fixed values such as 65 W CPU and 70 W T4 are used, describe them as an **assumed power proxy** or modeling assumption. Do not call them measured average runtime power or TDP-derived runtime power.

Where practical, NVIDIA power draw can be sampled from the runtime and reported as a proxy. Document sampling interval, aggregation rule, and limitations.

The basic energy relation $E = P \times t$ is acceptable when P is clearly identified as measured or assumed.

12. Cloud-cost corrections

Kaggle does not provide the project with an actual cloud billing measurement. Cost therefore must be modeled from an explicit provider/configuration/rate assumption.

- Select one documented cloud configuration for the cost model.
- Record provider, instance/GPU type, region if relevant, rate source, and access date.
- Label the result as modeled cloud cost, not actual Kaggle cost.
- Use the same costing basis when comparing CPU, GPU, and hybrid assignments.

13. Hybrid CPU/GPU execution corrections

Hybrid execution should be applied only to workloads that can actually be partitioned. “Hybrid” cannot simply mean running the same full workload twice.

13.1 Define a partition rule

The plan needs an explicit partition-ratio method. For example, a candidate ratio can be generated from predicted CPU/GPU throughput and then adjusted for transfer overhead and synchronization. The exact rule must be stated and tested.

Start with one technically partitionable workload as the hybrid proof-of-concept rather than claiming that all Rodinia workloads support hybrid execution.

14. Dependency/DAG corrections

Rodinia is a benchmark suite, not a universal task scheduler DAG. The project may construct a scheduling DAG from algorithmic stages or from a workload decomposition, but it must say clearly that this DAG is **project-created**.

For critical-path analysis, describe it as the longest-path estimate unless the implementation formally recomputes makespan after removing a node or edge. Avoid unsupported claims such as “the node is proven to determine makespan.”

14.1 Dependency count

dependency_count is useful as scheduler-level context, but separate intrinsic algorithmic dependencies from dependencies introduced by the project’s task decomposition.

15. Failure handling

A failed run should be preserved in the raw dataset with status/error information. It should not silently become a zero, NaN-imputed timing, or a valid aggregated observation.

- Raw layer: preserve return code, stderr/error class, parser status, and run status.
- Aggregation layer: use only valid comparable runs.
- ML layer: exclude invalid measurements from ground-truth label construction unless a failure itself is intentionally modeled as a target/context feature.

16. Dataset-size wording

Do not describe “10 workloads × 6 runs” as 60 independent ML samples. The six runs are repetitions of configurations. The meaningful ML sample count is the number of distinct workload/input/configuration instances after aggregation.

State the actual number of distinct valid workload/configuration records once profiling is complete. Do not promise a sample count before the profiling matrix is finished.

17. Baselines and evaluation

Baseline	Definition to use
CPU-only	All supported tasks assigned to CPU.
GPU-only	All GPU-supported tasks assigned to GPU.
Rule-based	A predeclared rule using comparable speedup/performance information and a fixed threshold.
Proposed scheduler	ML prediction + dependency/transfer/resource context + multi-objective decision.

Define “best-known assignment” operationally, for example as the best measured assignment among tested candidates for a given benchmark configuration. Do not call it globally optimal without proof.

Evaluate end-to-end metrics such as makespan/execution time, waiting time, energy proxy, modeled cost, utilization/resource pressure, scheduling overhead, and ML classification performance. Include per-workload and macro-level reporting.

18. Expected results → hypotheses

Replace definitive “expected results” with testable hypotheses. Examples:

- The adaptive scheduler may reduce execution time relative to fixed CPU-only/GPU-only policies on workloads where device suitability varies.
- Transfer-aware scheduling may reduce poor GPU assignments for transfer-heavy tasks.
- Dynamic objective weighting may change assignments when resource/energy/cost conditions change.
- Hybrid execution may improve performance for selected partitionable workloads, but is not expected to benefit every workload.

These are hypotheses to test, not findings to report in advance.

19. Corrected end-to-end architecture

```
Rodinia 3.1 workloads + benchmark data
↓
Workload-specific build + execution configuration
↓
Repeated CPU/GPU profiling
↓
Raw execution dataset
↓
Validated timing semantics + aggregation
↓
Aggregated workload performance dataset
↓
Pre-execution static/context features
↓
ML classifier (+ optional timing regression)
↓
Candidate CPU / GPU / Hybrid assignments
↓
Dependency + transfer + resource analysis
↓
Normalized multi-objective scoring
↓
Adaptive scheduling decision
↓
```

Actual execution
↓
Runtime measurements
↓
Feedback / model update

20. Recommended implementation order

1. Finish CFD source inspection and finalize timing semantics.
2. Correct CFD CPU/GPU parsers and rebuild/run configuration.
3. Finalize BFS timing semantics and remove any double-counting formula.
4. Create one robust workload-specific profiling record schema.
5. Profile the remaining common Rodinia workloads with explicit per-workload commands/parsers.
6. Generate the raw execution dataset and preserve failures.
7. Aggregate valid runs into workload/input performance records.
8. Design the leakage-free pre-execution ML feature table.
9. Construct preferred-device labels only for comparable measurements.
10. Train/evaluate a simple baseline classifier with workload-aware validation.
11. Add validated lookup or regression estimates for candidate execution times.
12. Implement the adaptive scheduler with dependency, transfer, and resource context.
13. Implement normalized weighted multi-objective scoring and measurable dynamic weights.
14. Add one hybrid proof-of-concept workload if technically partitionable.
15. Run CPU-only, GPU-only, rule-based, and proposed-scheduler comparisons.
16. Only then consider optional NSGA-II, richer regression, or additional ML models.

21. What does NOT need redesign

- The central project idea of adaptive CPU/GPU workload scheduling is sound.
- Using Rodinia 3.1 as the benchmark workload source is appropriate for the experimental design.
- Kaggle can be used as a cloud-hosted heterogeneous experimental environment, provided it is not described as a production multi-node cloud HPC cluster.
- The CPU/GPU/hybrid candidate set is appropriate.
- Dependency awareness, transfer awareness, dynamic objective weighting, and feedback are reasonable novelty components when implemented and measured precisely.
- The staged approach of simple ML first and advanced optimization later is preferable to adding complexity before data quality is established.

22. Final correction checklist

- ☐ CFD GPU parser matches actual output order.
- ☐ CFD CPU parser is based on actual CFD output, not BFS.
- ☐ CFD source timing semantics inspected and documented.
- ☐ BFS GPU total-path definition validated without double counting.
- ☐ BFS input size computed from actual file.
- ☐ Workload-specific commands/parsers stored explicitly.

- [] Raw, aggregated, and ML datasets separated.
- [] No post-execution timing leakage in pre-execution ML features.
- [] Expert-derived features have reproducible definitions.
- [] Preferred-device label created only for comparable timings.
- [] Workload-aware ML validation planned.
- [] Scheduler timing estimates have a defined source.
- [] Objective normalization specified.
- [] Resource-pressure proxy mathematically defined.
- [] Dynamic-weight triggers specified.
- [] Energy values labeled as measured or assumed proxies.
- [] Cloud costs have provider/rate/source/date assumptions.
- [] Hybrid partition ratio is explicitly defined and limited to suitable workloads.
- [] Project-created DAG is distinguished from Rodinia itself.
- [] Failures preserved in raw data and excluded from invalid aggregates.
- [] Baselines are defined consistently.
- [] Expected results are written as hypotheses.
- [] No optimality claims are made without proof.

23. Bottom line

Do not restart the project. The core architecture is usable. The immediate task is to make the profiling layer scientifically defensible. Once CFD and BFS timing semantics are locked down and the three-layer dataset is created, the ML and scheduler stages can proceed without the current leakage and circularity problems.

The most important principle for the next implementation steps is: **measure first, define semantics second, aggregate third, then build the predictor and scheduler from information genuinely available before execution.**